

Lab 11

Parser combinators in Haskell

Goals

In this lab you will learn to:

1. Understand parser combinators
2. Write your own parser combinators

Resources

Table 11.1: Lab Resources

Resource	Link
Functional parsing	https://youtu.be/dDtZLm7HIJs
Understanding parser combinators	https://fsharpforfunandprofit.com/posts/understanding-parser-combinators/
Understanding parser combinators: a deep dive - Scott Wlaschin	https://www.youtube.com/watch?v=RDalzi7mhdY

11.1 Defining basic parsers

Concept 11.1.1: Parser

A parser is a function that takes an unstructured input (a sequence of characters) and turns it into structured data (a parse tree), according to a set of rules.

We can translate this definition directly into the following type: `type Parser a = String -> a`.

In order to cover the case when the input doesn't have the desired form, we will define the following type to represent the parsing result:

Listing 11.1.1: ParseResult

Haskell code

```
data ParseResult a = Success a | Error String deriving (Show)
```

In the `Success` case, we return the parsed data, and in the `Error` case, we return an error message.

The updated definition for `Parser` now looks like: `type Parser a = String -> ParseResult a`.

Using these type definitions, we can implement a parser for the character "a":

Listing 11.1.2: parserA

Haskell code

```
parserA :: Parser Char
parserA "" = Error "End of input"
parserA (first:_) =
  if first == 'a' then
    Success 'a'
  else
    Error "Expected 'a'"
```

which can use as:

Haskell REPL

```
> parserA "abc"
Success 'a'
> parserA "xyz"
Error "Expected 'a'"
```

Note how the parser only returns the prefix that it recognized (i.e. the character `a`), ignoring the remaining input (i.e. the string `bc`). As title of the lab suggests, we would like to be able to combine parsers (i.e. use multiple simple parsers to make a more complex parser). Thus, for reasons that will become clear below, in the success case, we will also return the remaining input, giving us the updated definition of `ParseResult`:

Listing 11.1.3: Updated ParseResult

Haskell code

```
data ParseResult a = Success a String | Error String deriving (Show)
```

We will also need to make some changes to `parserA`:

Listing 11.1.4: Updated parserA

Haskell code

```

parserA :: Parser Char
parserA "" = Error "End of input"
parserA (first:rest) =
  if first == 'a' then
    Success 'a' rest
  else
    Error "Expected 'a'"

```

which will change the outputs of the examples to:

Haskell REPL

```

> parserA "abc"
Success 'a' "bc"
> parserA "xyz"
Error "Expected 'a'"

```

The final step to make our parsers more versatile is to define a new type instead of defining an alias: `data Parser a = Parser {runParser :: String -> ParseResult a}`. Having a new type instead of a type alias for a function will allow us to implement type classes for our parsers.

The final version of `parserA` will be:

Listing 11.1.5 of Simple.hs (parserA)

Haskell code

```

16 | parserA :: Parser Char
17 | parserA = Parser inner where
18 |   inner "" = Error "End of input"
19 |   inner (first:rest) =
20 |     if first == 'a' then
21 |       Success 'a' rest
22 |     else
23 |       Error "Expected 'a'"

```

Here we define a closure (the `inner` function) that will be wrapped by the `Parser` type. The `inner` function can directly access the input, which will be passed to the parser later.

To run the parser, we use the record field accessor function (`runParser`) to unwrap the function in the `Parser` type:

Haskell REPL

```

> runParser parserA "abc"
Success 'a' "bc"
> runParser parserA "xyz"
Error "Expected 'a'"

```

11.1.1 A more general parser

We can define a more general parser that can be “configured” from the outside to parse a character that satisfies a predicate¹:

¹In OOP terms, you can think of this as a “parser factory”

Listing 11.1.6 of Simple.hs (satisfies)

Haskell code

```

27 satisfies :: (Char -> Bool) -> Parser Char
28 satisfies predicate = Parser inner where
29   inner "" = Error "End of input"
30   inner (first:rest) =
31     if predicate first then
32       Success first rest
33     else
34       Error ("Unexpected character " ++ show first)

```

With this function we can create other parsers, such as `lower` and `upper`:

Listing 11.1.7 of Simple.hs (lower, upper)

Haskell code

```

38 lower :: Parser Char
39 lower = satisfies (\c -> elem c ['a'..'z'])
43 upper :: Parser Char
44 upper = satisfies (\c -> elem c ['A'..'Z'])

```

Exercise 11.1.1

Create a new parser using `satisfies`, `char :: Char -> Parser Char` that generates a parser that parses a given character (received as first argument).

Haskell REPL

```

> runParser (char 'a') "abc"
Success 'a' "bc"
> runParser (char 'x') "abc"
Error "Unexpected character 'a'"

```

Exercise 11.1.2

Create a new parser using `digit :: Parser Char` that parses a digit.

Haskell REPL

```

> runParser digit "123"
Success '1' "23"
> runParser digit "abc"
Error "Unexpected character 'a'"

```

Hints:

You might want to use (some of) the following functions: `satisfies`, `elem`.

11.2 Combining parsers

Parser combinators are all about creating new parsers from existing parsers *using combinator functions*.

11.2.1 Chaining (sequencing)

The first combinator is for chaining (sequencing) parsers, which we will call `andThen`:

Listing 11.2.1 of Simple.hs (andThen)

Haskell code

```

58 andThen :: Parser a -> Parser b -> Parser (a, b)
59 andThen pa pb = Parser inner where
60     inner input =
61         case runParser pa input of
62             Success a rest ->
63                 case runParser pb rest of
64                     Success b remaining -> Success (a,b) remaining
65                     Error err -> Error err
66             Error err -> Error err

```

Haskell REPL

```

> runParser (andThen lower upper) "aBc"
Success ('a', 'B') "c"
> runParser (andThen lower upper) "abc"
Error "Unexpected character 'b'"

```

`andThen pa pb` tries to run parser `pa`, if it succeeds, it runs the second parser `pb` on the remaining input from `pa`. If both parsers succeed, their results are returned in a tuple.

We can use `andThen` to create a parser for a given string, expressed as a series of character parsers:

Listing 11.2.2 of Simple.hs (string)

Haskell code

```

70 string :: String -> Parser String
71 string "" = Parser (\input -> Success "" input)
72 string (c:cs) = Parser inner where
73     inner "" = Error "End of input"
74     inner input =
75         case runParser (andThen (char c) (string cs)) input of
76             Success (p, ps) rest -> Success (p:ps) rest
77             Error err -> Error err

```

We first try to parse the first character using the `char` parser, then if it succeeds, we parse the rest of the string recursively.

Haskell REPL

```

> runParser (string "ab") "abc"
Success "ab" "c"
> runParser (string "ab") "xy"
Error "Unexpected character 'x'"

```

11.2.2 Alternatives (choice)

The second important combinator is for choosing between multiple combinators, which we will call `orElse`:

Listing 11.2.3 of Simple.hs (orElse)

Haskell code

```

81 | orElse :: Parser a -> Parser a -> Parser a
82 | orElse pa pb = Parser inner where
83 |     inner input =
84 |         case runParser pa input of
85 |             Success a rest -> Success a rest
86 |             Error _ -> runParser pb input

```

`orElse pa pb` tries to run the first parser `pa`. If `pa` succeeds, it returns its result, else it runs `pb` on the original input.

Haskell REPL

```

> runParser (orElse (char 'a') (char 'x')) "xy"
Success 'x' "y"
> runParser (orElse (char 'a') (char 'x')) "by"
Error "Unexpected character 'b'"

```

We can use `orElse` to create a parser for lowercase or uppercase letters, by just reusing the `lower` and `upper` parsers:

Listing 11.2.4 of Simple.hs (letter)

Haskell code

```

90 | letter :: Parser Char
91 | letter = lower `orElse` upper

```

11.2.3 Transformation

The third important combinator is for transforming the results of a parser, which we will call `pMap`²:

Listing 11.2.5 of Simple.hs (pMap)

Haskell code

```

95 | pMap :: (a -> b) -> Parser a -> Parser b
96 | pMap f p = Parser inner where
97 |     inner input =
98 |         case runParser p input of
99 |             Success r rest -> Success (f r) rest
100 |             Error err -> Error err

```

`pMap f p` transforms the result of a `Parser a` into `Parser b`. For example, it can be used to transform a `Parser String` into a `Parser Bool`:

Listing 11.2.6 of Simple.hs (bool)

Haskell code

```

104 | bool :: Parser Bool
105 | bool = pMap read (string "True" `orElse` string "False")

```

Exercise 11.2.1

*

Create a new parser for strings, `string'`, that only uses the combinators defined so far (`andThen`, `orElse` and `pMap`), without using inner functions.

²If only there was a typeclass that dealt with applying a mapping function to a data structure so we could implement it here ...

11.2.4 Repetition

The final combinators are utility functions for matching a parser zero or more times (`many`) or one or more times (`some`):

Listing 11.2.8 of Simple.hs (`many`)

Haskell code

```
112 many :: Parser a -> Parser [a]
113 many p = Parser inner where
114   inner "" = Success [] ""
115   inner input =
116     case runParser p input of
117       Success r rest ->
118         case runParser (many p) rest of
119           Success rs remaining -> Success (r:rs) remaining
120           Error _ -> Success [] input
```

Listing 11.2.9 of Simple.hs (`some`)

Haskell code

```
124 some :: Parser a -> Parser [a]
125 some p = Parser inner where
126   inner "" = Error "End of input"
127   inner input =
128     case runParser p input of
129       Success r rest ->
130         case runParser (many p) rest of
131           Success rs remaining -> Success (r:rs) remaining
132           Error err -> Error err
```

Note that since zero matches is also a valid input for `many`, it never fails!

Haskell REPL

```
> runParser (many (char 'a')) "aaabc"
Success "aaa" "bc"
> runParser (many (char 'a')) "bc"
Success "" "bc"
> runParser (some (char 'a')) "aaabc"
Success "aaa" "bc"
> runParser (some (char 'a')) "bc"
Error "Unexpected character 'b'"
```

Exercise 11.2.2

*

Create a new parser `number :: Parser Int` that parses a number.

Haskell REPL

```
> runParser number "123"
Success 123 ""
> runParser number "123abc"
Success 123 "abc"
> runParser number "abc"
Error "Unexpected character 'a'"
```

Hints:

Use the `some`, `pMap` combinators with the `digit` parser.

Use the `read` function to obtain an integer from the string.

11.3 Practice problems

Exercise 11.3.1

*

Create a new combinator `pRepeat :: Int -> Parser a -> Parser [a]` that tries to run the given parser `n` times on the input.

Haskell REPL

```
> runParser (pRepeat 1 (char 'a')) "aa"
Success "a" "a"
> runParser (pRepeat 3 (char 'a')) "aaaa"
Success "aaa" "a"
> runParser (pRepeat 3 (char 'a')) "aa"
Error "Unexpected end of input"
```

Exercise 11.3.2

*

Create a new combinator `pThen :: Parser a -> Parser b -> Parser b`. First, it runs the first parser, `pa`. If `pa` succeeds, it discards the result and runs `pb` on **the remaining input from `pa`**.³ If `pa` fails, the error is returned.

Haskell REPL

```
> runParser (pThen (char ' ') (char 'a')) " abc"
Success 'a' "bc"
> runParser (pThen (many (char ' ')) (some lower)) "  abc"
Success "abc" ""
> runParser (pThen (many (char ' ')) (some lower)) "abc123"
Success "abc" "123"
> runParser (pThen (many (char ' ')) (some lower)) " BC123"
Error "Unexpected character 'B'"
```

Try to solve the problem in 2 ways:

- By creating a new parser, based on `andThen`
- By only using existing parsers (`andThen` and `pMap`)

In the previous lab, a certain type class had a function with a very similar definition...

Exercise 11.3.3

*

Create a new combinator `try :: Parser a -> Parser (Maybe a)` which tries to run the given parser, returning the result wrapped in `Just` if it succeeds, or `Nothing` if it fails. `try` always succeeds.

Haskell REPL

```
> runParser (try digit) "1"
Success (Just '1') ""
> runParser (try digit) "a"
Success Nothing "a"
```


Try to solve the problem in 2 ways:

- By creating a new parser (i.e. with an inner function)
- By only using existing combinators (i.e. `pMap`, `orElse`, `andThen`, etc.)

Exercise 11.3.4

**

Create a parser `sepBy1 sep p` with the signature `sepBy1 :: Parser a -> Parser b -> Parser [b]`. It alternates between `p` and `sep`, starting and ending with `p` and returning the outputs of `p` in a list.

```
Haskell REPL
> runParser (sepBy1 (char ',') (many alpha)) "a,b,c"
Success ["a", "b", "c"] ""
> runParser (sepBy1 (char ',') (many alpha)) "abc"
Success ["abc"] ""
> runParser (sepBy1 (char ',') (many alpha)) "a,,b"
Success ["a"] ",,b"
> runParser (sepBy1 ws (many alpha)) "a b c d"
Success ["a", "b", "c", "d"] ""
> runParser (sepBy1 (char ',') (many alpha)) ""
Error "End of input"
```

Exercise 11.3.5

**

Create a parser `sepBy sep p` with the signature `sepBy :: Parser a -> Parser b -> Parser [b]`. It alternates between `p` and `sep`, starting and ending with `p` and returning the outputs of `p` in a list. Like `many`, `sepBy` always succeeds.

```
Haskell REPL
> runParser (sepBy (char ',') (many alpha)) "a,b,c"
Success ["a", "b", "c"] ""
> runParser (sepBy (char ',') (many alpha)) "abc"
Success ["abc"] ""
> runParser (sepBy (char ',') (many alpha)) "a,,b"
Success ["a"] ",,b"
> runParser (sepBy ws (many alpha)) "a b c d"
Success ["a", "b", "c", "d"] ""
> runParser (sepBy (char ',') (many alpha)) ""
Success [] ""
```

Exercise 11.3.6

**

Create a new combinator `between :: Parser a -> Parser b -> Parser c -> Parser c` which takes as arguments 3 parsers: `between pHd pTl p`. It runs them in the order `pHd`, `p` and `pTl`, passing the remaining input from the one to the other. If all 3 parsers succeed, it discards the results from `pHd` and `pTl` and returns the result of `p`. If any parser fails the error is returned.

```
Haskell REPL
> runParser (between (char '[') (char ']') (string "abc")) "[abc]xyz"
Success "abc" "xyz"
```

Hints:

Think about how can you use `andThen` to chain 3 parsers. What is the type of the returned parser? How can you *transform* this type to match the required return type `(Parser c)`?

Is there any function that already does part of this work?

Exercise 11.3.7

**

Create a new combinator `pWith :: Parser a -> (a -> Parser b) -> Parser b`. First, it runs the first parser, `pa`. If `pa` succeeds, it uses the result from `pa` to obtain the second parser, `pb`, then it runs `pb` on the remaining input from `pa`.⁴ If `pa` fails, the error is returned.

```
Haskell REPL
> runParser (pWith (char 'a') (char)) "aa"
Success 'a' ""
> runParser (pWith (number) (\n -> pRepeat n (char 'a')) "2aa"
Success "aa" ""
> runParser (pWith (number) (\n -> pRepeat n (char 'a')) "3aaaaa"
Success "aaa" "aa"
> runParser (pWith (number `andThen` letter) (\ (n, c) -> pRepeat n (char c)) "2bbba"
Success "bb" "ba"
```

It seems like the behavior of a certain type class just keeps appearing...

Exercise 11.3.8

**

Create a new parser `ws` for consuming whitespace. It should consume zero or more space ' ' or newline '\n' characters.

```
Haskell REPL
> runParser ws "abc"
Success "" "abc"
> runParser ws "  abc"
Success "  " "abc"
```

Exercise 11.3.9

**

Given the type definition `data Date = Date {year :: Int, month :: Int, day :: Int}`, create a new parser `date` for the following date format: YYYY-MM-DD.

```
Haskell REPL
> runParser date "2024-01-01"
Success (Date {year = 2024, month = 1, day = 1}) ""
> runParser date "2024-01"
Error "End of input"
> runParser date "2024/01/01"
Error "Unexpected character '/'"
```